Relatório – Tarefa 2: Grafo de palavras de 4 letras

Disciplina: TEG (2026/2) · Linguagem: C (C11) · Ambiente: Ubuntu

1. Identificação da equipe

Integrante	Nome	Matrícula
1	Nicolas Oliveira Veiga	2061012546
2	Guilherme Wippel Kormann	2061012535

- Curso / turma / professor: Ciência da Computação · CCI251-04U · Gilmário Barbosa
- Repositório no GitHub: https://github.com/nic-veiga/trabalho-teg.git
- Arquivo de entrega: NicolasV_GuilhermeW.zip

2. Identificação da tarefa

Tarefa 2 – Implementação de um grafo com carga primária a partir de um arquivo de base.

O objetivo é construir um grafo não direcionado e ponderado G(V,E) em que os vértices são palavras em português de 4 letras, lidas de uma base fornecida no Moodle (`entrada.txt`), e em que dois vértices p e q são ligados por uma aresta quando as palavras diferem em exatamente uma posição. A ideia é estudar a transformação de uma palavra em outra, trocando uma letra por vez. Exemplo do enunciado: LAGO → LOGO → LEGO → CEGO → CEGA → CENA.

Requisitos do enunciado e onde cada um foi atendido:

Requisito	Como foi atendido	Onde
(a) Implementação em C	C11, usando somente a biblioteca padrão	`src/*.c`
(b) Carga primária do grafo a partir da base	Leitura, validação, vértices e arestas	`grafo.c`, função `grafo_carregar`
(c-i) Componentes conexos, tamanhos, palavra central (grau máximo) e de menor conexão (grau mínimo)	BFS iterativa e cálculo dos graus por componente	`analise.c`, funções `analisar_componentes` e `imprimir_componentes`
(c-ii) Grafo simples ou multigrafo, com contagem de laços e arestas múltiplas	Inspeção das listas de adjacência	`analise.c`, função `verificar_tipo_grafo`
(c-iii) Sequência de arestas entre duas palavras fornecidas pelo usuário, com Dijkstra, e comparação	Dijkstra com dois critérios de custo	`dijkstra.c` e função `consultar` em `main.c`

Todos os itens são acessíveis pelo menu interativo do programa (seção 3.4) e também pela linha de comando.

3. Explicação conceitual da solução

3.1 Entrada

- **Arquivo de base (`entrada.txt`):** uma palavra por linha. O arquivo fornecido tem 1604 linhas, todas com 4 letras ASCII (sem acentos), finais de linha no padrão Windows (CRLF) e sem quebra de linha após a última palavra.
- **Formas de uso:** o programa tem duas interfaces. Sem argumentos (`./teg`), abre um **menu interativo**; com argumentos, funciona pela **linha de comando**: `./teg [opções] [arquivo [origem destino]]`. Na linha de comando, se o arquivo não for informado, o programa usa `entrada.txt`; no menu, a opção 1 propõe `entrada.txt` (basta apertar Enter).
- **Consulta do usuário:** duas palavras de 4 letras (origem e destino), digitadas em um laço interativo (no menu ou na linha de comando) ou passadas diretamente na linha de comando. Maiúsculas e minúsculas na digitação são indiferentes. Uma linha vazia ou a palavra `sair` encerra o laço.
- **Tratamento da leitura:** o programa remove CR, LF e espaços das pontas, ignora um eventual BOM UTF-8 e linhas em branco, e descarta (contando no relatório) linhas que não tenham exatamente 4 letras. Linhas muito longas são descartadas sem estourar o buffer.

3.2 Modelagem

O grafo é representado por **listas de adjacência**. Cada vértice guarda a palavra (chave, a forma usada nas comparações), a grafia original (exibição), o grau e o vetor de arestas; cada aresta guarda o vértice de destino e o peso. A escolha por listas se justifica pela esparsidade: com 1590 vértices e 7249 arestas, o grau médio é 9,12 e a densidade é de apenas 0,57%. Uma matriz de adjacência teria cerca de 2,5 milhões de células para guardar poucas ligações.

Peso das arestas. O enunciado pede um grafo ponderado, mas não define o peso. Adotamos a distância alfabética entre as duas letras que diferem (por exemplo, a→e vale 4 e o→a vale 14). Uma diferença só de caixa (Rosa/rosa, no modo estrito) vale 1, para que todo peso seja no mínimo 1.

3.3 Processo

Passo 1 – Carga primária. O programa lê o arquivo linha a linha, valida e normaliza cada palavra, ordena todas com `qsort` e elimina as repetições, mantendo a grafia da primeira ocorrência. Com os vértices em ordem alfabética, a busca de uma palavra digitada é uma busca binária, O(log V). Em seguida, para cada par de vértices (i, j) com i < j, compara-se as 4 letras; se exatamente uma difere, cria-se a aresta nas listas dos dois vértices, já com o peso.

Passo 2 – Componentes conexos. Usamos busca em largura iterativa (fila em vetor, sem recursão, para não correr risco de estourar a pilha). O próprio vetor da fila agrupa os vértices de cada componente. Para cada componente são calculados o tamanho, o grau máximo (palavra central) e o grau mínimo (palavra de menor conexão). Os componentes são ordenados do maior para o menor e, em caso de empate de grau, as palavras empatadas são listadas em ordem alfabética (até 4, seguidas de `(+N)`).

Passo 3 – Simples ou multigrafo. Em vez de assumir a resposta, o programa inspeciona as listas de adjacência: ordena os destinos de cada vértice e procura destinos iguais ao próprio vértice (laços) e destinos repetidos (arestas paralelas). Também confere que a soma dos graus é igual a 2·|E|. Pela regra

do enunciado o grafo é necessariamente simples, pois uma palavra difere de si mesma em 0 posições (nunca em 1), e dois vértices distintos são ligados por no máximo uma aresta.

Passo 4 – Dijkstra. Implementado com heap binário de mínimo e remoção preguiçosa (entradas obsoletas do heap são ignoradas ao serem retiradas). O algoritmo mantém vetores de distância, predecessor e vértices fechados, encerra assim que o destino é fechado e reconstrói o caminho pelos predecessores. Como o enunciado pede para comparar sequências de arestas, o Dijkstra é executado duas vezes sobre o mesmo grafo:

- **[A] Custo unitário** (cada aresta custa 1): obtém o menor número de transformações.
- **[B] Custo ponderado** (cada aresta custa o seu peso): obtém a menor soma de distâncias alfabéticas.

O peso total do caminho A também é calculado, para que os dois caminhos possam ser comparados nos dois critérios.

Complexidade de cada etapa:

Etapa	Custo
Busca de uma palavra digitada (busca binária)	O(log V)
Construção das arestas (comparação par a par, 4 letras por par)	$O(V^2)$
Componentes conexos (BFS iterativa, sem recursão)	O(V + E)
Simples × multigrafo	O(E log grau)
Dijkstra (heap binário, remoção preguiçosa)	O((V + E) log V)

3.4 Interface: menu interativo

Quando executado sem argumentos (ou com `-m` / `--menu`), o programa abre um menu em laço. O programa mantém em memória o **estado da sessão** (grafo carregado, componentes, classificação simples/multigrafo e estatísticas da carga), de modo que a base é carregada e analisada uma única vez e depois reaproveitada pelas demais opções.

Opção	Ação	Seção do relatório / requisito
1	Carregar base de palavras (pede o arquivo; Enter usa `entrada.txt` ou o arquivo já carregado)	Seção 1 · requisito (b)
2	Resumo da carga (vértices e arestas)	Seção 1
3	Componentes conexos (tamanhos, grau máximo e mínimo)	Seção 2 · requisito (c-i)
4	Grafo simples ou multigrafo?	Seção 3 · requisito (c-ii)

Opção	Ação	Seção do relatório / requisito
5	Caminho entre duas palavras (Dijkstra), em laço até uma linha vazia	Seção 4 · requisito (c-iii)
6	Relatório completo (seções 1, 2 e 3)	Seções 1 a 3
7	Alternar o modo de comparação (padrão ⇄ estrito) e recarregar a base	Decisão de projeto 3
0	Sair	–

Comportamentos importantes do menu:

- As opções 2 a 6 exigem uma base carregada; sem ela, o programa avisa para usar a opção 1 primeiro.
- Se o carregamento de um arquivo falhar (arquivo inexistente, sem palavras válidas ou falta de memória), a base anterior é mantida.
- A opção 7 recarrega o mesmo arquivo no novo modo; se a recarga falhar, o modo anterior é mantido. Sem base carregada, apenas troca o modo para a próxima carga.
- O fim da entrada (Ctrl+D) encerra o programa liberando a memória, assim como a opção 0.

3.5 Saída

O programa imprime quatro seções (no menu, cada uma pode ser pedida separadamente, conforme a tabela da seção 3.4):

1. **Carga primária:** arquivo, modo de comparação, linhas lidas/válidas/ignoradas, palavras repetidas fundidas, $|V|$ e $|E|$.
2. **Componentes conexos:** quantidade e, para cada componente, tamanho, palavra(s) central(is) com o grau máximo e palavra(s) de menor conexão com o grau mínimo. Componentes de uma só palavra aparecem como “isolada”.
3. **Simples ou multigrafo:** número de laços, de arestas múltiplas, verificação da soma dos graus e a conclusão.
4. **Caminho entre duas palavras:** componente e grau de origem e destino; as sequências [A] e [B] com o peso de cada aresta, o número de arestas e o peso total de cada uma; e uma comparação em texto (mesma sequência, critérios que divergem, mesmo peso com menos arestas, mesmo número de arestas com menos peso, ou sequências diferentes de custo igual). Se as palavras estiverem em componentes diferentes, ou se alguma não constar na base, o programa informa.

3.6 Decisões de projeto

1. **Peso das arestas:** distância alfabética (item 3.2), pois o enunciado não define o peso.
2. **Comparação de sequências:** Dijkstra com dois critérios de custo (item 3.3), uma interpretação possível de “encontre e compare as sequências”.
3. **Maiúsculas e minúsculas:** a base tem 14 palavras que aparecem duas vezes, só com a caixa trocada (por exemplo, Rosa/rosa, Roma/roma, Para/para). Por padrão elas são fundidas em um único vértice (1604 linhas → 1590 vértices). A opção `-s` (modo estrito) ou a opção 7 do menu trata cada grafia como um vértice distinto (1604 vértices). Como a grafia exibida é a da primeira

ocorrência, alguns caminhos mostram palavras com inicial maiúscula (por exemplo, Rosa → … → Nova).

4. **Interface:** o menu interativo é o comportamento padrão (sem argumentos), por ser mais simples de operar; o modo de linha de comando foi mantido para consultas diretas e é o usado pela suíte de testes automatizados.

4. Instruções de compilação

- **IDE / arquivo de projeto:** não foi usada IDE nem arquivo de projeto.
- **Linha de comando:** a compilação é feita com `gcc` e `make`.
- **Bibliotecas:** nenhuma biblioteca externa foi carregada. O código usa apenas a biblioteca padrão de C (`stdio.h`, `stdlib.h`, `string.h`, `limits.h`) e não precisa de `-lm`.
- **Ambiente de desenvolvimento e teste:** Ubuntu 24.04 com gcc 13.3.

Instalação das ferramentas (se necessário):

```
sudo apt install build-essential
```

Compilação e execução:

Comando	O que faz
`make`	gera o executável `./teg`
`./teg`	abre o menu interativo
`./teg -m`	idem (força o menu)
`./teg entrada.txt`	linha de comando: carrega, imprime as seções 1 a 3 e pergunta pares de palavras em laço
`./teg entrada.txt lago cena`	seções 1 a 3 e consulta direta entre duas palavras
`./teg -n entrada.txt lago cena`	idem, sem o relatório dos itens 1 a 3
`./teg -s entrada.txt`	modo estrito (maiúsculas distintas)
`./teg --ajuda`	lista as opções

Compilação manual, sem make:

```
gcc -std=c11 -O2 -o teg src/*.c
```

O Makefile compila com `-std=c11 -Wall -Wextra -Wpedantic -O2`, e o código compila sem nenhum aviso. Alvos extras: `make test` (testes automatizados, equivalente a `bash tests/run_tests.sh`) e `make debug` (compila com AddressSanitizer e UBSan). No modo interativo, uma linha vazia ou a palavra `sair` encerra a consulta de caminhos; no menu, a opção 0 encerra o programa.

Atalho para Windows. O arquivo `executar.bat` apenas entra na pasta do projeto, executa `teg.exe` (que abre o menu) e pausa ao final. É uma conveniência para quem usa Windows: o executável precisa ter sido gerado antes com gcc, e o ambiente de avaliação (Ubuntu) não depende desse arquivo.

Estrutura dos arquivos entregues:

```
.
├── Makefile
├── README.md
├── executar.bat        # atalho para Windows (executa teg.exe)
├── entrada.txt
├── src/
│   ├── main.c          # linha de comando, menu interativo, relatório e consulta
│   ├── grafo.h/.c      # estruturas, carga, arestas, busca
│   ├── analise.h/.c    # componentes conexos, simples x multigrafo
│   └── dijkstra.h/.c   # heap binário e Dijkstra
└── tests/run_tests.sh
```

5. Descrição e análise de resultados

5.1 Carga do grafo

Das 1604 linhas da base, todas eram válidas (0 ignoradas), mas 14 eram repetições diferindo apenas na caixa. Resumo nos dois modos:

Métrica	Modo padrão	Modo estrito (-s)
Linhas lidas / válidas / ignoradas	1604 / 1604 / 0	1604 / 1604 / 0
Palavras repetidas fundidas	14	0
Vértices (V)	1590	1604
Arestas (E)	7249	7216
Componentes conexos	28	40
Maior componente	1561 vértices	1562 vértices

No modo padrão, o grau médio é 9,12 e a densidade é de 0,57%. O grau mais frequente é 7 (140 palavras). A palavra de maior grau é *caia*, com 26 vizinhos (única com esse grau); 25 palavras têm grau 0 e 39 têm grau 1.

No modo estrito o número de arestas é menor porque palavras que só diferem na caixa deixam de se fundir e, ao mesmo tempo, passam a ser comparadas letra a letra com a caixa original.

5.2 Componentes conexos

No modo padrão há 28 componentes conexos:

Componente	Quantidade	Vértices	Detalhes
Gigante	1	1561 (98,2% das palavras)	Palavra central: *caia* (grau 26); 35 palavras com grau mínimo 1
Pares	2	2 cada	broa–proa e IMPA–impo

Componente	Quantidade	Vértices	Detalhes
Isolados	25	1 cada	palavras para as quais nenhuma outra da base difere em uma única letra

As 25 palavras isoladas são: azul, Baal, bits, CNPq, Dion, Eden, Egeu, emir, Esau, Etna, expo, flor, hoje, icou, Jean, noel, Ogum, pneu, psiu, siri, UFRJ, Utah, util, York e zebu. Chama a atenção que 12 delas são nomes próprios ou siglas.

A escolha entre os modos mostra como a definição de “mesma palavra” afeta a conectividade. No modo estrito há 40 componentes: o gigante (1562 vértices), 3 pares (Marx–Mary, Unix–onix e broa–proa) e 36 palavras isoladas. O par IMPA–impo, por exemplo, só existe no modo padrão: com a caixa original, IMPA e impo diferem em quatro posições.

5.3 Simples ou multigrafo

O programa encontrou 0 laços e 0 arestas múltiplas, e a soma dos graus (14498) é igual a $2 \cdot |E|$. Portanto, o grafo é simples, como previsto pela definição: nenhuma palavra difere de si mesma em exatamente uma posição, e dois vértices distintos são ligados por, no máximo, uma aresta.

5.4 Caminhos com Dijkstra

Resultados obtidos com o programa (A = custo unitário, B = custo ponderado):

Par	A: arestas / peso	B: arestas / peso	Observação
mata → mala (exemplo do enunciado)	1 / 8	1 / 8	mesma sequência
lago → cena	4 / 34	4 / 34	sequências diferentes, mesmo custo
casa → vida	4 / 42	5 / 42	mesmo peso; A usa 1 aresta a menos
rosa → nova	3 / 29	3 / 7	mesmo número de arestas; B pesa 22 a menos
amor → odio	6 / 48	6 / 36	mesmo número de arestas; B pesa 12 a menos
ilha → geou (par mais distante do componente gigante)	14 / 98	15 / 80	critérios divergem de fato
flor → lago	–	–	sem sequência: *flor* é palavra isolada

Sequências conhecidas:

- **lago → cena:** A = lago, cago, caga, cana, cena; B = lago, lego, cego, cega, cena. Há mais de um caminho mínimo e o Dijkstra devolve um deles. A sequência de 5 arestas citada no enunciado é válida, mas não é a mais curta.
- **casa → vida:** A = casa, cada, ceda, veda, vida; B = casa, cara, dara, dira, vira, vida.
- **rosa → nova:** B = Rosa, rota, nota, Nova.
- **ilha → geou:** é o caso em que os critérios realmente divergem: A tem uma aresta a menos, mas B pesa 18 a menos.
- **flor → lago:** o programa informa que as palavras estão em componentes diferentes.

Interpretação. O critério unitário responde "qual o menor número de trocas de letra?", que é a leitura clássica da transformação de palavras. O critério ponderado favorece trocas entre letras próximas no alfabeto (como casa → cara, peso 1) e por isso pode aceitar mais arestas para pagar menos. Nos pares testados, as sequências foram diferentes em 5 de 6 casos conectados, e B foi estritamente mais leve em 3 deles. Isso mostra que a escolha do critério muda a resposta, e que a comparação pedida no enunciado é informativa.

O grafo também tem a estrutura de um "mundo pequeno": o diâmetro do componente gigante é 14 arestas (par ilha–geou, confirmado pelo programa), mas a distância média entre duas palavras conectadas é de apenas 5,4 arestas (valor obtido com um script auxiliar de conferência).

5.5 Validação e desempenho

- **Testes automatizados:** `make test` executa 40 verificações (totais do grafo, componentes, tipo do grafo, Dijkstra, entrada do usuário, modo estrito, arquivos de entrada com defeitos e códigos de saída) com o programa compilado com `-Werror`, AddressSanitizer e UBSan.
- **Conferência independente:** os resultados (`|V|`, `|E|`, número e tamanho dos componentes e custos do Dijkstra) foram comparados com um script auxiliar em Python, em 400 pares de palavras sorteados (200 por modo), sem nenhuma divergência. Esse script não faz parte da entrega.
- **Sensibilidade dos testes:** introduzimos dois defeitos de propósito (aresta para palavras com 2 diferenças e Dijkstra ponderado ignorando o peso), e a suíte os detectou.
- **Entradas defeituosas:** foram testados arquivo com BOM, CRLF, linhas em branco, linhas inválidas, uma linha de 3000 caracteres e última linha sem quebra, além de arquivo inexistente e arquivo vazio (erro com código de saída 1).
- **Desempenho:** a carga, a análise e uma consulta levam cerca de 10 ms com a base fornecida.

5.6 Limitações e possíveis melhorias

- A construção das arestas compara todos os pares, $O(V^2)$. Para 1590 palavras é instantâneo, mas para bases muito maiores seria melhor agrupar as palavras por "padrão com uma letra oculta" (por exemplo, `l*go`), o que reduz o custo para algo próximo de $O(V \cdot k)$, com $k = 4$.
- O peso é uma escolha nossa, já que o enunciado não o define. Outras definições (por exemplo, frequência da palavra) levariam a caminhos diferentes.
- Só são aceitas letras ASCII; palavras acentuadas seriam descartadas, o que não afeta a base fornecida, que não tem acentos.
- Quando existem vários caminhos mínimos, o Dijkstra retorna apenas um (o desempate é determinístico, pelo índice do vértice).

- A suíte automatizada exercita o modo de linha de comando; o menu interativo não tem testes automatizados próprios, embora use as mesmas funções de carga, análise e consulta.

Conclusão

O programa carrega a base, constrói o grafo de 1590 palavras com 7249 arestas, identifica o componente gigante (1561 palavras) e os outros 27 componentes (2 pares e 25 palavras isoladas, somando 29 palavras), confirma que o grafo é simples e encontra, via Dijkstra, as sequências de transformação entre duas palavras quaisquer, comparando o menor número de trocas com a menor soma de pesos. Todas as funcionalidades podem ser usadas por um menu interativo ou pela linha de comando, e os resultados foram verificados por testes automatizados e por uma conferência independente.